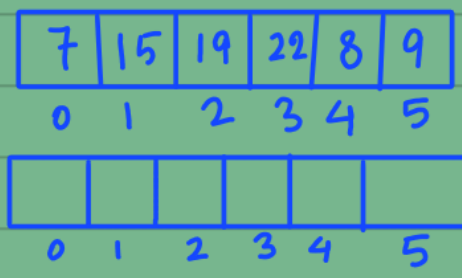
* Stack Operation while implementing Using Array :-




```
struct stack{
    int size;
    int top;
    int *arr;
}
```

using pointers

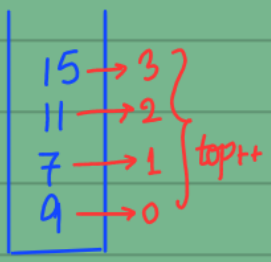
```
struct stack *s
s->size;
s->top;
s->arr;
```

Creating a stack

```
struct size s;
s.size;
s.top;
s.arr;
```

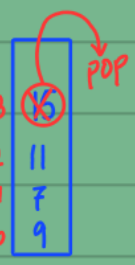
Op 1: Push (nothing is returned)

```
struct stack *s;
s->size = 8;
s->top = -1;
s->arr = (int*) malloc(s->size * sizeof(int));
if (isFull(s)){
    Stack overflow
}
else{
    s->top++
    s->arr[s->top] = val
}
```

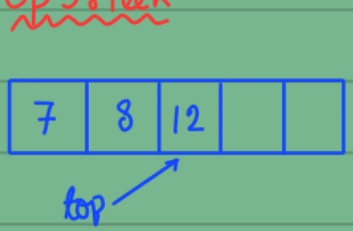


Op 2: Pop

```
if (isEmpty(s)){
    stack underflow;
    return -1;
}
else{
    int val = s->arr[s->top];
    s->top = s->top-1;
    return val;
}
```



Op 3: Peek





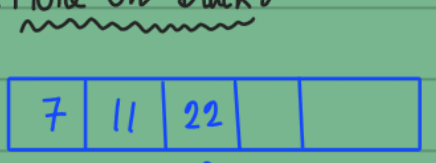
top = 2

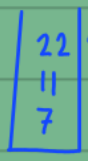
position(i)	Array index
1	2 ⇝ (top - i + 1)
2	1
3	0

```
int peek(struct stack *s, int pos){
    if(s->top - i + 1 < 0){
        Not valid;
        return -1;
    }
    else{
        return s->arr[s->top - i + 1];
    }
```

(pos: not index)

* More On Stack :-





Time Complexity → stackBottom always O(1)

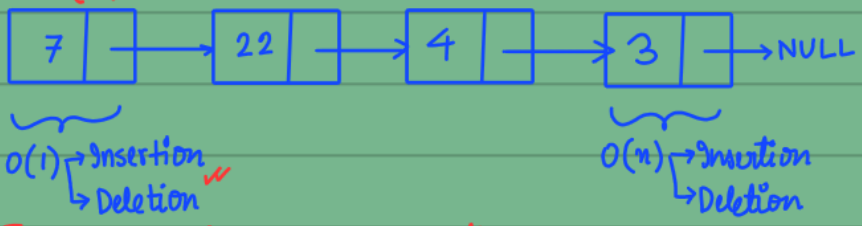
stackTop always O(1)

stackTop → return s->arr[s->top];

stackBottom → return s->arr[0];

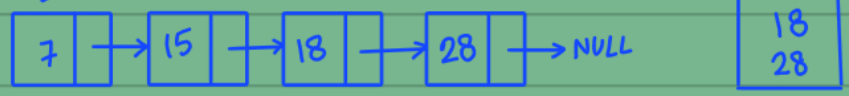
- isEmpty → O(1)
- isFull → O(1)
- Pop → O(1)
- Push → O(1)
- peek → O(1)

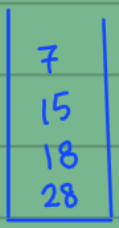
* Implementing Stack using linked-list :-




```
struct Node {
    int data;
    struct Node *next;
};
```

1> This side used for push & pop operation
2> 'head' is now → top
3> stack Empty Cond^n → (top == NULL)
4> stack full cond^n → heap memory is exhausted ⇝ (ptr == NULL)
(theoritically not full)
5> We can always set custom size.





1> isEmpty():

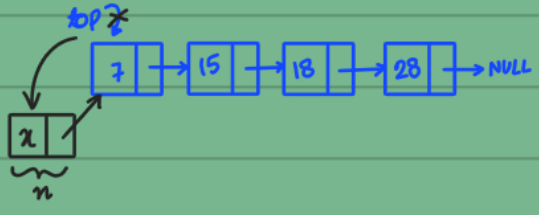
```
int isEmpty;
if(top == NULL){
    return 1;
}
else return 0;
```

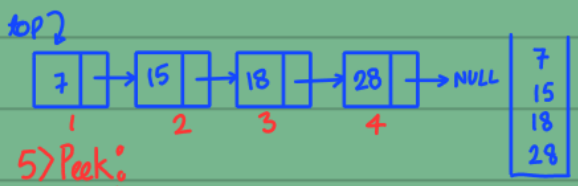
2> isFull():

```
struct Node *n = (struct Node*) malloc (sizeof (struct Node))
if (n == NULL){ // Memory cannot get allocated dynamically
                ↳ heap full
    return 1;
}
else return 0;
```

3> Push :

```
struct Node *n = (struct Node*)...
if(n == NULL){
    stack overflow
}
else{
    n->data = data;
    n->next = top;
    top = n;
```





4> Pop:

```
if (isEmpty){
    stack underflow
}
else{
  struct Node* n = tp;
  top = tp -> next;
  int x = n -> data;
  free(n);
  return x;
}
```

using top as global variable

5> Peek:

```
peek(int pos){
  struct Node*ptr = top;
  for(i=0;(i<pos-1&& ptr!=NULL);i++){
     ptr = ptr -> next;
  }
  if(ptr != NULL){
     return ptr->data;
  }
  else{
     return -1;
  }
```

pos	no of times movement
1	0
2	1
3	2
4	3

6> Stacktop:-

```
int stacktop(){
  return top->data;
}
```

7> Stackbottom:-

```
int stackbottom(struct Node *ptr){
   while (ptr!=NULL){
      ptr=ptr->next;
   }
   return ptr->data;
}
```

* Parenthesis Matching:-

{(3*2)-(8-2)} → Balanced



Exp = 3 * 2 - (8 + 1)

(→ if yes → push into the stack

) → if yes → pop out of the stack

- Condition for balanced exp:-

① while popping → stack should not underflow ⇒ if it happens (unbalanced)

② At EOE, stack must be empty ⇒ otherwise unbalanced

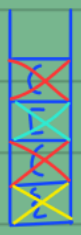
```
int parenthesesMatch(char *exp){
   struct stack *sp;
   //create the stack
   for(i=0; exp[i]!='\0'; i++){
      if(exp[i]=='('){
         push(sp,exp[i]);
      }
      else if(exp[i]==')'){
         if(isEmpty(sp)){
            return 0;
         }
         pop(sp);
      }
   }
   if(isEmpty(sp)){
      return 1;
   }
   else return 0;
```

* Multiple Parentheses Checking:-

exp = {7-(3-2)+[8*(9+1)]}

op = {[(→ push

cl = }])→ pop



cl = }]) → pop

Check pop succes

Yes → keep checking → is Empty? → Yes → Balanced / No → Khel khatam

No → Not balanced

* Infix, Prefix & Postfix:-

→ notation to write an expression

1. Infix → a+b, a/b, a-b, x-y
   operand <operator> operand

2. Prefix → +ab, -xy, *pq, /ab
   <operator> opr^n1 opr^n2

3. Postfix: ab+, pq*, xy-
   op^1 op^2 <operator>

Infix: A*(B+C)*D

Postfix: ABC+*D*

operator	precedence
()	3
*, /	2
+, -	1

Q1. x-y*z to prefix and postfix?

x-y*z

Postfix: (x-(y*z)) = (x-(yz*)) = xyz*-

Prefix: (x-(y*z)) = (x-(*yz)) = -x*yz

Q2. p-q-r/a

((p-q)-(r/a))

Postfix: ((pq-)-(ra/)) = pq-ra/-

Prefix: ((-pq)-(/ra)) = --pq/ra

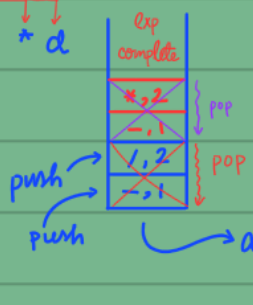
Q3. (m-n)*(p+q) to postfix?

{(mn-)*(pq+)} = mn-pq+*

* Infix to Postfix using Stack:-

x - y / z - 1 * d

opr	Precedence
*, /	2
+, -	1



→ at last stack empty

Postfix exp:- x-(yz/)-(kd*)

(xyz/-)-(kd*)

= xyz/-kd*-

* Code for infix to postfix conversion:-

infix = | a | - | b | * | d | + | c | \0 |

postfix = | a | b | d | * | - | c | + | \0 |

```
char * intoPost (char *infix){
    stouct stack *sp; // init the stack
    char * postfix =(chas *) malloc (strlen (infix+1) * sizeof (char));
    int i=0; // infix scanner
    int j=0; // postfix fill
    while (infix[i] !='\0'){
        if (! isoperator (infix[i])){
            postfix[j] = infix[i];
            i++; j++;
        }
        else {
            if (prece (infix[i])>prece (stackTop(sp))){
                push (sp, infix[i]);
            }
            else{
                postfix[j] = pop(sp);
                j++;
            }
            postfix[j] ='\0';
            return postfix;
        }
    }
```

* Queue Data Structure:-

→ FIFO (First In First Out)

* Insertion End & Deletion End Different

Queue ADT

Data: ① Storage
② Ins. End
③ Del. End

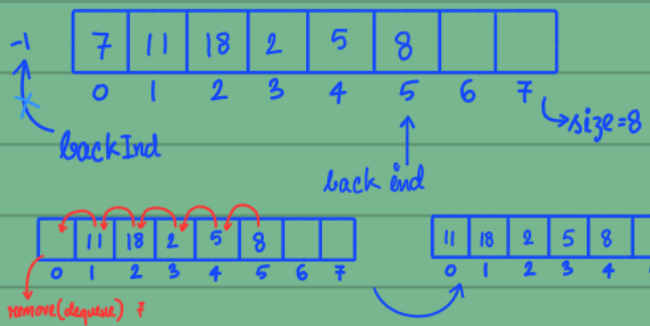
Methods: ① enqueue (in)
② dequeue (out)
③ first val
④ Last val
⑤ peek(pos)
⑥ isEmpty
⑦ isFull

NOTE → Queue not limited to counters
Queue can be implemented various ways.
① using arrays
② using linkedlist
③ using other ADTs.

* Implementation of Queue :-

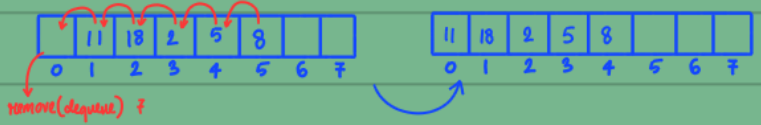
(enqueue)

• Insert : increment backInd
insert at backInd

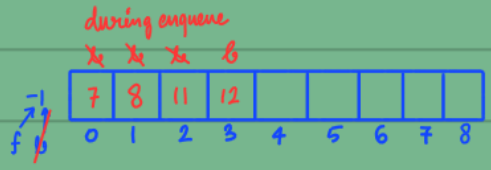


(dequeue)

• Deletion : remove element at ind 0
shift all elements
↳ O(n)



frontInd → f , backInd → b

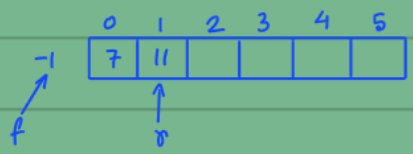




[first element = frontInd+1
last element = backInd]

- Queue empty → frontInd = backInd
- Queue full → backInd = size-1

* Queue Using Arrays :-




```
struct queue{
    int size;
    int f;
    int r;
    int *arr;
};
```

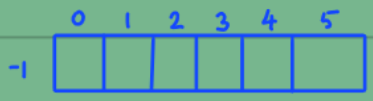
```
int main(){
    struct queue q;
    q.size=10;
    q.r=q.f=-1;
    q.arr=(int *)malloc(q.size*sizeof(int));
}
```

```
void enqueue(struct queue *q, int value){
    if(isFull(q)){
        Queue Overflow
    }
    else{
        q->r=q->r+1;
        q->arr[q->r]=value;
    }
}
```

```
int dequeue(struct queue *q){
    int a=-1; // For some reason if there is any prim returning the dequeue value it will return -1.
    isEmpty() { if(q->f==q->r){
                    No element to dequeue
                }
    else{
        q->f++;
        a=q->arr[q->f];
    }
    return a;
}
```

```
int isFull(struct queue *q){
    if(q->r==q->size-1){
        return 1;
    }
    else return 0;
```

* Circular Queue :-



* Drawbacks of Queue using array

1) Space is not used efficiently

Linear increment

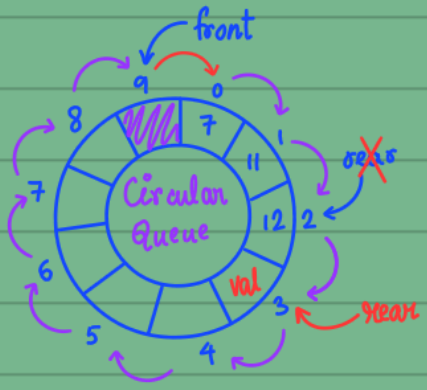
0 → 1 → 2 → 3 → 4 → 5

$i = i + 1$

Circular increment

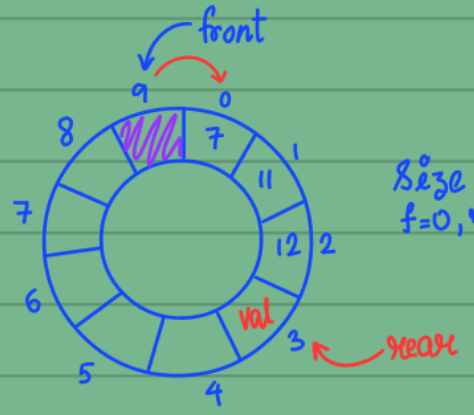
0 → 1 → 2 → 3 → 4 → 5 → 0

$i = (i+1) \% size$




```
void enqueue(struct queue *q, int val){
    if((q->r+1)%q->size == q->f){
        Queue Overflow
    }
    else{
        q->r = (q->r+1)%q->size;
        q->arr[q->r] = val;
    }
}
```

```
int dequeue(struct queue *q){
    if(q->r == q->f){
        Queue Empty
    }
    else{
        q->f = (q->f+1)%q->size;
        val = q->arr[q->f];
    }
    return val;
}
```



Size = 10
f = 0, r = 0 } starting

that's why it is wrong

if f = -1, r = -1

↓ insert

r = 0 → → → r = q

↓

r = 0

↓

but f stays at -1, a/c to this Queue is never gonna fill

after this elements will get overwritten

* Queue Using Linked Lists :-



isEmpty → f=NULL

isFull → n=NULL

```
void enqueue(struct node *front,
             struct node *rear, int val){
  Struct node *n = (struct node*) malloc(sizeof
                                (struct node));
  if (n==NULL){
      printf("Queue full");
  }
  else{
     n->data = val;
     n->next = NULL;
     if(f==NULL){
        f=r=n;
     }
     r->next = n;
     r=n;
  }
```

special case

```
int dequeue(struct Node *f){
  int val = -1;
  Struct node *ptr = f;
  // Check for Queue empty
  else block {
     f = f->next;
     val = ptr->data;
     free(ptr);
     return val;
  }
}
```

* Double Ended Queue (DEQueue):-

Queue	DEQueue
1. Insertion from rear	1. Insertion from Rear & Front
2. Deletion from front	2. Deletion from Rear & Front

• Types of DEQueue :-

1. Restricted Input DEQueue → Insertion from front not allowed
2. Restricted Output DEQueue → Deletion from rear not allowed

• Operation :-

1. isEmpty
2. isFull
3. Traversal
4. enqueueFront
5. enqueueRear
6. dequeueFront
7. dequeueRear